

FATIH SULTAN MEHMET VAKIF UNIVERSITESI

Software Verification, Validation & Security Testing

DEFECT REPORT

Smart Hospital Appointment System

Team: ZeroDay Squad | Project: TestHub | Date: April 2026

Saed O. S. Radi | Anas I. A. Alhatti | Abdulrahman Aljabahji | Ahmed Alsaleh

1. Defect Summary

This report documents all defects discovered during testing of the Smart Hospital Appointment System. Each defect includes the violated system requirement, steps to reproduce, severity, priority, and resolution status.

3 out of 4 defects have been resolved. DEF-004 remains open due to implementation complexity.

Total	High	Medium	Closed	Open	Affected TCs	Exec. Rate	Pass Rate
4	2	2	3	1	6 Test Cases	88/88 (100%)	93.2%

2. Defect Details

DEF ID	Title	Requirement Violated	Steps to Reproduce	Expected vs Actual	Severity	Status	Affected TCs
DEF-001	CSRF Protection Disabled	System MUST validate CSRF tokens on all state-changing POST requests to prevent cross-site request forgery.	1. Open SecurityConfig.java 2. Find securityFilterChain 3. Observe: .csrf(AbstractHttpConfigurer::disable)	Expected: CSRF validated. Actual: CSRF completely disabled.	High	Closed	TC-SEC-009
DEF-002	Appointment Booking Always Fails	System MUST allow patients to book appointments and store with PENDING status.	1. Login as patient1 2. Click Software Eng dept 3. Click Book Appointment 4. Select date and time 5. Click Confirm Booking	Expected: Booking confirmed. Actual: 'Failed to book. Please try again.' for all doctors.	High	Closed	TC-FUNC-010 TC-FUNC-028
DEF-003	Admin Create Patient NullPointerException	System MUST allow admins to create patient accounts without server errors.	1. Login as testadmin 2. Go to Manage Users 3. Fill Create Patient form 4. Click Create 5. Observe error	Expected: Patient created. Actual: HTTP 500 NullPointerException at AdminController.java:41.	Medium	Closed	TC-FUNC-013
DEF-004	No Brute Force Protection on Login	System MUST implement brute force protection — logout or delay after 5 consecutive failed attempts.	1. Open login page 2. Enter patient1 / wrongpass 3. Click LOGIN 10 times 4. Enter correct password 5. Observe — login succeeds	Expected: Account locked after 5 failures. Actual: Unlimited attempts. Login succeeds immediately.	Medium	Open	TC-SEC-004 TC-SEC-013

3. Detailed Defect Descriptions

DEF-001: CSRF Protection Disabled

Defect ID	DEF-001
Title	CSRF Protection Disabled in Security Configuration
Requirement Violated	System MUST validate CSRF tokens on all state-changing POST requests to prevent cross-site request forgery attacks.
Related Test Cases	TC-SEC-009
Module	Security / Authentication
Severity	High
Priority	High
Status	Closed ✓
Description	CSRF protection is explicitly disabled in SecurityConfig.java using .csrf(AbstractHttpConfigurer::disable). The application does not validate CSRF tokens on state-changing requests, making it vulnerable to CSRF attacks where malicious websites can trigger actions on behalf of authenticated users.
Steps to Reproduce	1. Open SecurityConfig.java in IntelliJ IDEA 2. Find the securityFilterChain method 3. Observe: .csrf(AbstractHttpConfigurer::disable)
Expected Result	CSRF tokens validated on all POST, PUT, DELETE requests. Cross-site requests rejected without valid token.
Actual Result	CSRF protection completely disabled. Any cross-site POST request accepted without token validation.
Evidence	TC-SEC-009-FAIL.png — Screenshot of SecurityConfig.java showing .csrf(AbstractHttpConfigurer::disable)
Fix Applied	Removed .csrf(AbstractHttpConfigurer::disable) from SecurityConfig.java and enabled Spring Security default CSRF protection.

DEF-002: Appointment Booking Always Fails

Defect ID	DEF-002
Title	Appointment Booking Feature Fails for All Doctors
Requirement Violated	System MUST allow patients to successfully book appointments with available doctors and store the booking record in the appointments table with PENDING status.

Related Test Cases	TC-FUNC-010 (FAIL), TC-FUNC-028 (FAIL)
Module	Patient / Appointment Booking
Severity	High
Priority	High
Status	Closed ✓
Description	When a patient attempts to book an appointment with any available doctor, the system consistently returns 'Failed to book. Please try again.' The appointments table remains empty after all booking attempts, confirming the booking is never processed by the backend.
Steps to Reproduce	1. Login as patient1 / Patient@1234 2. Click Software Eng department 3. Click Book Appointment for Dr. Salem or Dr. Ahmed 4. Select any future date and time 5. Click Confirm Booking 6. Observe error message
Expected Result	Appointment booked. Record appears in appointments table with PENDING status. Visible in patient Medical History.
Actual Result	'Failed to book. Please try again.' error displayed. No appointment record created. SELECT * FROM appointments returns 0 rows.
Impact	Core feature non-functional. TC-FUNC-028 also failed as duplicate booking prevention cannot be tested without a working booking feature.
Evidence	TC-FUNC-010-FAIL.png — Screenshot showing 'Failed to book. Please try again.' error
Fix Applied	Identified and fixed API endpoint mismatch between React frontend and Spring Boot backend. Booking endpoint URL corrected. Appointments now saved successfully in the database.

DEF-003: Admin Create Patient Causes NullPointerException

Defect ID	DEF-003
Title	NullPointerException When Admin Creates Patient via Dashboard
Requirement Violated	System MUST allow admins to create new patient accounts from the Admin Dashboard with all required fields properly submitted to the backend.
Related Test Cases	TC-FUNC-013
Module	Admin / User Management
Severity	Medium
Priority	Medium
Status	Closed ✓
Description	When Admin attempts to create a new Patient from the Admin Dashboard, the backend throws NullPointerException at AdminController.java:41. The password field is received as null due to a mismatch between the frontend form field name and the expected backend parameter.
Steps to Reproduce	1. Login as testadmin / Admin@1234 2. Go to Admin Dashboard > Manage Users 3. Fill Create Patient form: Full Name, Username, Password 4. Click Create 5. Observe Internal Server Error
Expected Result	New patient account created and visible in Users List with PATIENT role.
Actual Result	HTTP 500 Internal Server Error. Console: NullPointerException: Cannot invoke String.matches(String) because User.getPassword() is null at AdminController.java:41
Root Cause	Frontend React form sends password field under a different name than expected by AdminController @ModelAttribute binding.
Evidence	TC-FUNC-013-FAIL.png — Screenshot of Internal Server Error in Admin Dashboard
Fix Applied	Corrected form field name in React Admin Dashboard to match @ModelAttribute parameter in AdminController.java. Added null check at line 41 as a safety measure.

DEF-004: No Brute Force Protection on Login

Defect ID	DEF-004
Title	Login Endpoint Has No Brute Force or Rate Limiting Protection

Requirement Violated	System MUST implement brute force protection — account lockout or response delay MUST be applied after 5 consecutive failed login attempts per user or IP address.
Related Test Cases	TC-SEC-004 (FAIL), TC-SEC-013 (FAIL)
Module	Authentication / Security
Severity	Medium
Priority	Medium
Status	Open X
Description	The login endpoint accepts unlimited consecutive failed attempts without any rate limiting, account lockout, delay, or CAPTCHA challenge. Even after 10+ failed attempts, the correct password is accepted immediately with no lockout. This makes the system vulnerable to automated password guessing attacks.
Steps to Reproduce	1. Open https://localhost:8080/login 2. Enter patient1 / wrongpass123 3. Click LOGIN 10 times consecutively 4. On 11th attempt enter correct password: Patient@1234 5. Observe: login succeeds immediately — no lockout triggered
Expected Result	After 5 failed attempts: account temporarily locked OR increasing delay added OR CAPTCHA shown. Correct password rejected during lockout period.
Actual Result	TC-SEC-004: System accepted 10+ attempts with no lockout, delay or CAPTCHA. TC-SEC-013: Correct password accepted immediately even after 10+ failures. No lockout mechanism exists.
Evidence	TC-SEC-004-FAIL.png — Screenshot showing repeated failed attempts accepted with no protection
Resolution Note	Fix was attempted using Spring Security custom authentication provider and login attempt tracking. Due to circular dependency issues between Spring beans and framework limitations, a complete lockout implementation could not be achieved. Defect remains open for future resolution.